

[10강] Agentic AI & Summary

조현수

이화여자대학교 인공지능학과 조교수

저작권 안내

(주)업스테이지가 제공하는 모든 교육 콘텐츠의 지식재산권은

운영 주체인 (주)업스테이지 또는 해당 저작물의 적법한 관리자에게 귀속되어 있습니다.

콘텐츠 일부 또는 전부를 복사, 복제, 판매, 재판매 공개, 공유 등을 할 수 없습니다.

유출될 경우 지식재산권 침해에 대한 책임을 부담할 수 있습니다.

유출에 해당하여 금지되는 행위의 예시는 다음과 같습니다.

- 콘텐츠를 재가공하여 온/오프라인으로 공개하는 행위
- 콘텐츠의 일부 또는 전부를 이용하여 인쇄물을 만드는 행위
- 콘텐츠의 전부 또는 일부를 녹취 또는 녹화하거나 녹취록을 작성하는 행위
- 콘텐츠의 전부 또는 일부를 스크린 캡처하거나 카메라로 촬영하는 행위
- 지인을 포함한 제3자에게 콘텐츠의 일부 또는 전부를 공유하는 행위
- 다른 정보와 결합하여 Upstage Education의 콘텐츠를 알아볼 수 있는 저작물을 작성, 공개하는 행위
- 제공된 데이터의 일부 혹은 전부를 Upstage Education 프로젝트/실습 수행 이외의 목적으로 사용하는 행위

강의 목표

학습 목표

- 지금까지 다룬 Prompt Engineering의 핵심 개념과 기법을 정리하는 시간을 갖는다.
- Agentic AI의 개념과 특징을 간략히 이해한다.

01. Prompt Engineering Summary

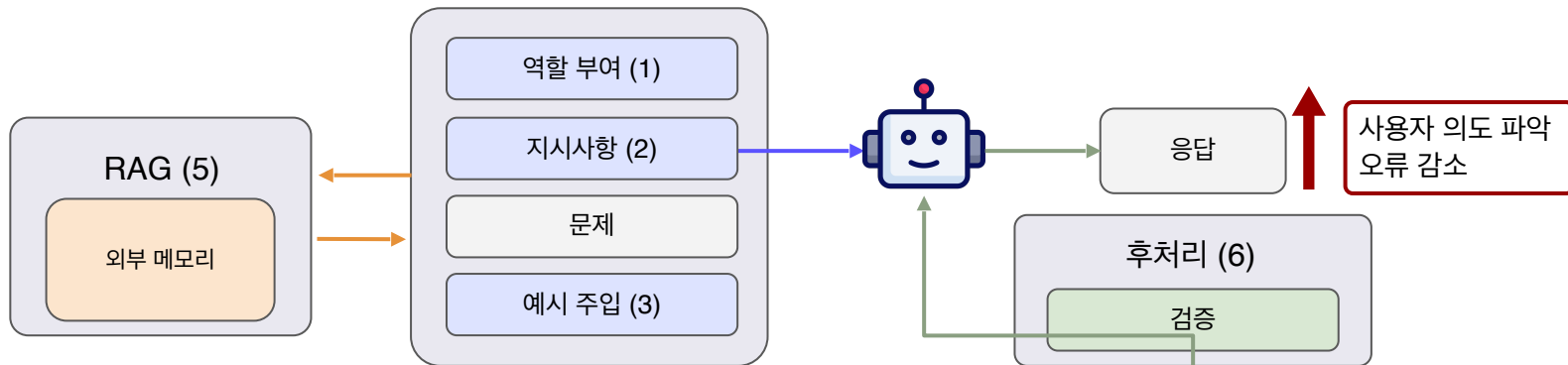
02. Agentic AI

(1) 기존 AI의 한계

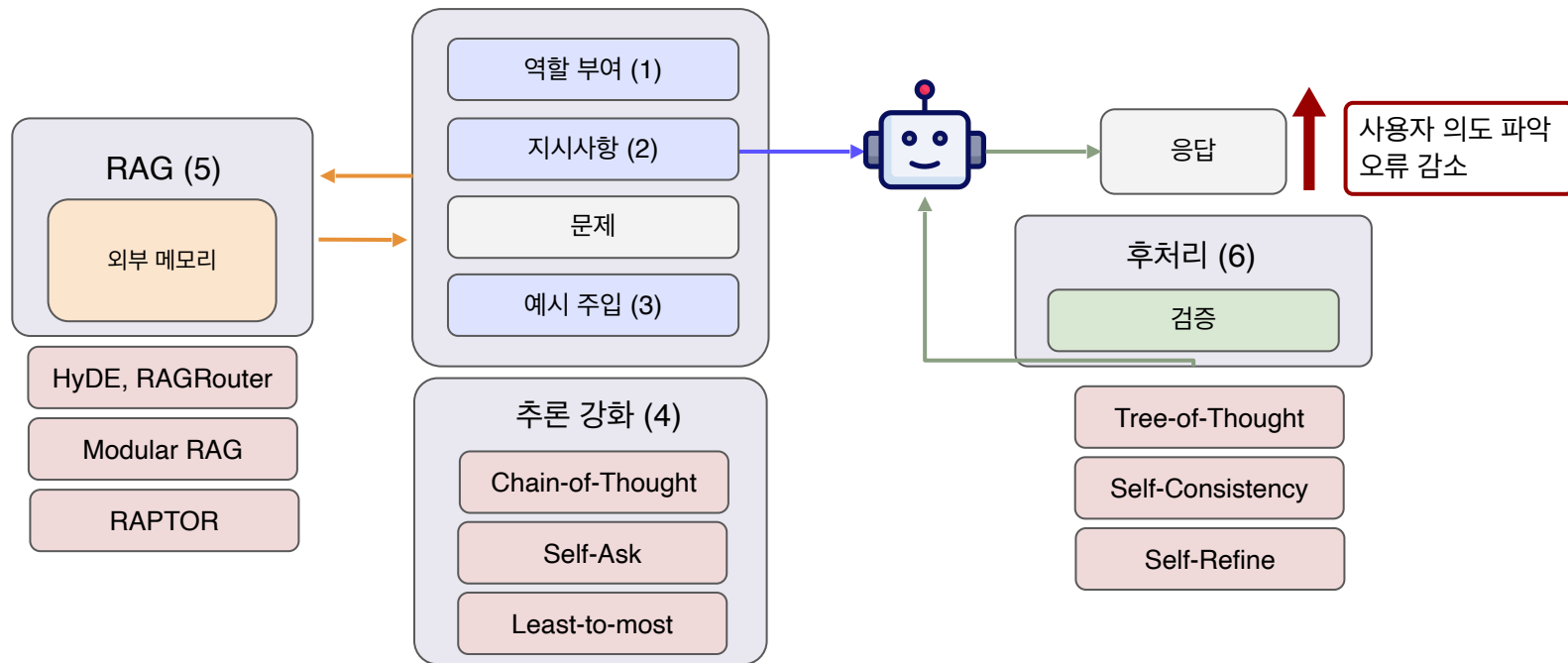
Prompt Engineering Summary

1~9강까지 배운 내용에 대한 Recap

Prompt Engineering



Prompt Engineering



Prompt Engineering

(1) 역할 부여 - Persona Injection

LLM에 대해서 역할(role)을 부여

- Role Prompting: "너는 20년 차 시니어 개발자야" 등 역할 부여.
- Emotional Prompting: 감정적인 호소나 동기 부여를 통해 답변의 품질을 미세 조정
(예: "이건 내 커리어에 정말 중요한 일이야").

```
{  
  "model": "solar-pro",  
  "messages": [  
    { "role": "system", "content": "당신은 법률 문서 분석을 수행하는 보조 AI입니다." },  
    { "role": "user", "content": "다음 계약서를 핵심 조항 위주로 요약해줘." },  
    { "role": "assistant", "content": "계약서 요약은 다음과 같습니다." }  
  ]  
}
```



Prompt Engineering

(2) 지시사항

사용자가 원하는 요구사항을 명시적으로 넣어주는 것

ex. 말할 때의 말투 (개조식, 두괄식으로 작성, 극존칭 사용), 구체적인 행동, 제약 조건, 스타일, 형식 등을 정의하는 것이 해당

예시

Model Input

Q: 사과가 빨간색인 이유는 뭐야?
과학적인 근거를 대서 설명해줘

Model Output

사과가 붉은색을 띠는 이유는 안토시아닌(anthocyanin)이라는 색소 때문입니다. 사과는 처음에는 엽록소 때문에 초록색을 띠지만, 익어가 는 과정에서 엽록소가 분해되고 안토시아닌이 생성되면서 붉은색으 로 변하게 됩니다. ...

Model Input

Q: 사과가 빨간색인 이유는 뭐야?
아이들의 동심을 유지할 수 있도록 설명해줘

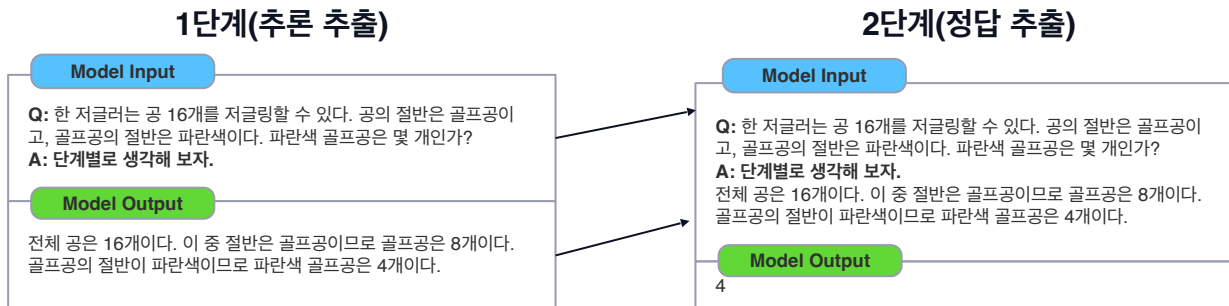
Model Output

사과도 여름에 우리가 햇볕 아래서 신나게 놀면 얼굴이 빨갱게 타는 거랑 비슷해.
처음에는 나뭇잎 뒤에 숨어있는 초록색 아기 사과였는데, 매일매일 따 뜻한 햇님을 듬뿍 받으면서 무럭무럭 자란 거지. 그러다가 가을이 와 서 '나 이제 아주 달콤하게 잘 익었어요!' 하고 자랑하고 싶어서 얼굴 이 발그레하게 빨개진 거란다

Prompt Engineering

(2) 지시사항 - Zero-shot-CoT prompting

- 1단계: 추론 추출
 - 프롬프트 구성 : 입력(질문) + 연쇄적 사고를 끌어내기 위한 트리거 문장
 - 출력 : 추론 과정을 담은 문장
- 2단계: 정답 추출
 - 프롬프트 구성 : 1단계 입력 + 1단계 출력 + 정답 추출용 트리거 문장
 - 출력 : 최종 답변



Prompt Engineering

(2) 지시사항 - Plan-and-Solve(PS) prompting

- 지시사항에 “트리거 문장”을 추가하여 LLM이 문제를 두 단계에 걸쳐 해결(문제 계획 수립+하위 과제 수행)하도록 유도하는 방법
 - 트리거 문장 예시 : 먼저 문제를 이해하고 해결을 위한 계획을 세운 다음, 그 계획을 단계별로 수행하자
- Zero-shot CoT의 단계 누락 문제를 해결
- Zero-shot Co과 동일하게 2단계로 진행

Zero-shot CoT

Model Input

Q: 학생이 20명인 댄스 수업에서 20%는 컨템포러리 댄스를 수강했고, 남은 학생 중 25%는 재즈 댄스를 수강했으며, 나머지는 힙합 댄스를 수강했다. 전체 학생 중 힙합 댄스를 수강한 학생의 비율은 몇 퍼센트인가?

A: 단계별로 생각해 보자.

Model Output

먼저, 전체 학생의 20%가 컨템포러리 댄스를 수강했으므로 80%가 남는다. 다음으로, 남은 80%의 25%가 재즈 댄스를 수강했는데, 이는 원래 전체 학생의 25%가 재즈 댄스를 수강했다는 의미가 된다. 그러면 원래 전체의 55%가 남는다. 마지막으로, 남은 학생들(55%)이 힙합 댄스를 수강했다. 따라서 전체 학생의 55%가 힙합 댄스를 수강했다. ❌(추론과정 오류)

PS

Model Input

Q: 학생이 20명인 댄스 수업에서 20%는 컨템포러리 댄스를 수강했고, 남은 학생 중 25%는 재즈 댄스를 수강했으며, 나머지는 힙합 댄스를 수강했다. 전체 학생 중 힙합 댄스를 수강한 학생의 비율은 몇 퍼센트인가?

A: 먼저 문제를 이해하고 문제를 해결하기 위한 계획을 세워보자. 그런 다음 계획에 따라 단계별로 문제를 해결하자.

Model Output

계획:

1단계: 컨템포러리 댄스와 재즈 댄스를 수강한 학생 수의 합을 계산한다.

2단계: 힙합 댄스를 수강한 학생 수를 계산한다.

3단계: 힙합 댄스를 수강한 학생의 비율을 계산한다.

해결:

1단계: 20명의 20%는 4명이다. 남은 16명의 25%는 4명이다. 따라서 컨템포러리 댄스와 재즈 댄스를 수강한 학생은 총 8명이다.

2단계: 남은 12명은 힙합 댄스를 수강했다.

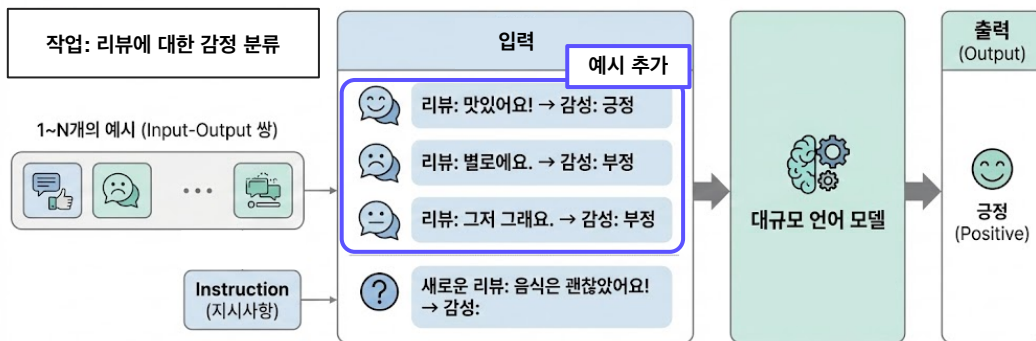
3단계: 힙합 댄스를 수강한 학생의 비율은 $12/20 = 60\%$ 이다. ✓

Prompt Engineering

(3) 예시 주입: In-Context Learning (=Few-shot Prompting / Few-shot Learning)

Instruction과 함께 1~N개의 예시를 제공 (Input-Output 쌍) ex. '감성 분석', '정규 표현식 추출' 등의 복잡한 작업에 대해서 주로 사용

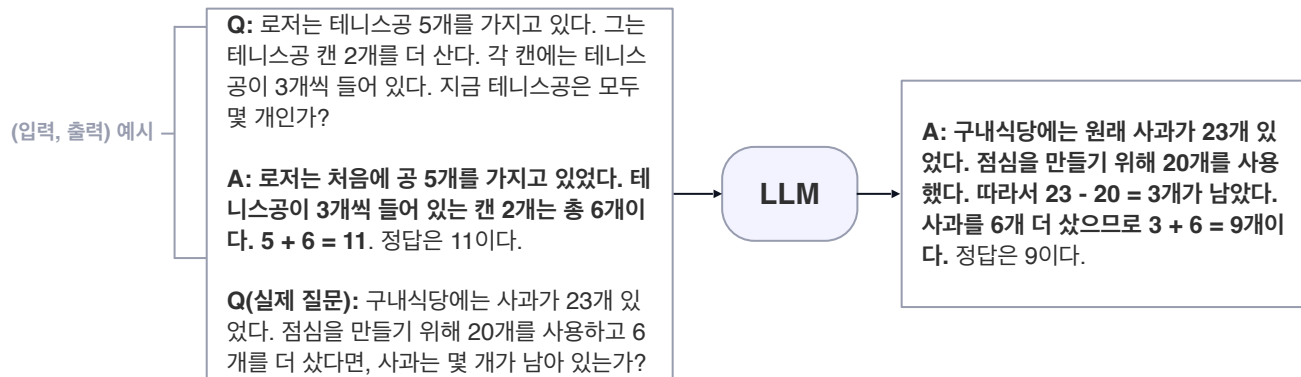
- 장점: 정확도 및 성능 향상, **Formatting**에도 효과적 (output을 제시하고 있으므로 모델이 output 형식을 모방)
- 단점: zero-shot보다 많은 token 소모, 비용 증가



Prompt Engineering

(4) 추론 강화: Chain-of-Thought(CoT) Prompting

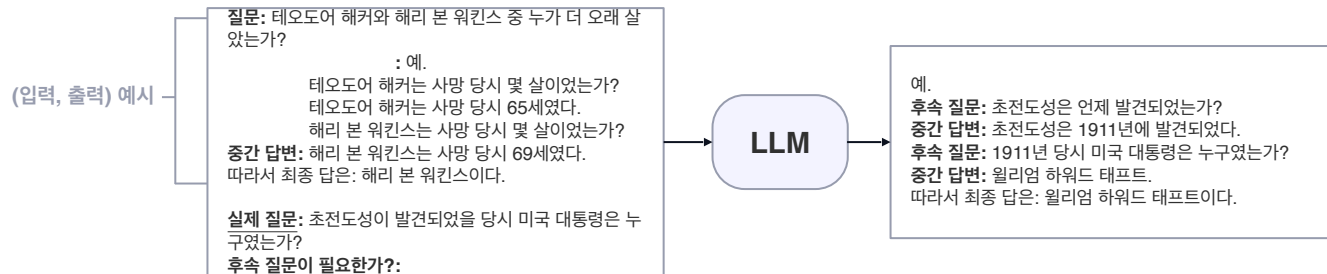
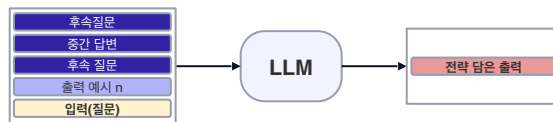
- 예시에 중간 추론 과정을 추가하여 CoT 생성을 유도하는 방법
 - Chain-of-Thought(CoT) : 최종 출력으로 이어지는 일련의 자연어 중간 추론 과정
- 프롬프트 구성 : (입력, CoT를 포함한 출력) 예시 + 입력(질문)



Prompt Engineering

(4) 추론 강화: Self-Ask

- 예시를 통해 모델 스스로 **후속 질문**과 그에 대한 **답변**을 생성한 뒤 **최종 답변**을 출력하도록 유도하는 방법
- CoT와 달리, 모델이 최종 답변을 내놓기 전까지의 과정이 **구조적임**(후속 질문, 중간 답변으로 구분)
- CoT와 동일하게, 후속질문-중간답변 그리고 **최종 답변까지의 과정이 모델에 의해 자동으로 진행**
- 프롬프트 구성 : (입력, 후속질문 필요 여부, 후속질문, 중간답) 예시 + 입력(질문) + 후속질문 필요 여부
 - 후속질문 필요 여부 문구 : Are follow up questions needed here:

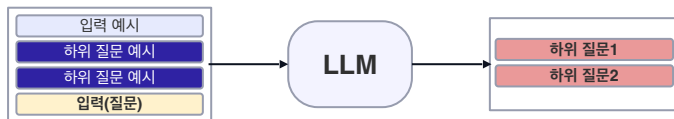


Prompt Engineering

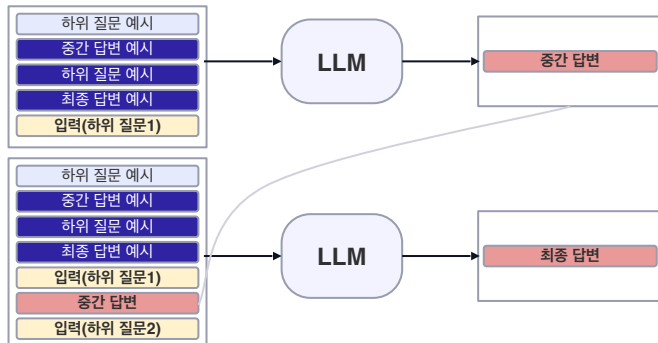
(4) 추론 강화: Least-to-most prompting

- 각 단계마다 예시를 제공(few-shot prompting)하여 문제 분해(1단계) 및 문제 순차 해결(2단계)을 유도하는 방법

단계 1: 질문을 하위 질문으로 분해



단계 2: 하위 질문을 순차적으로 해결

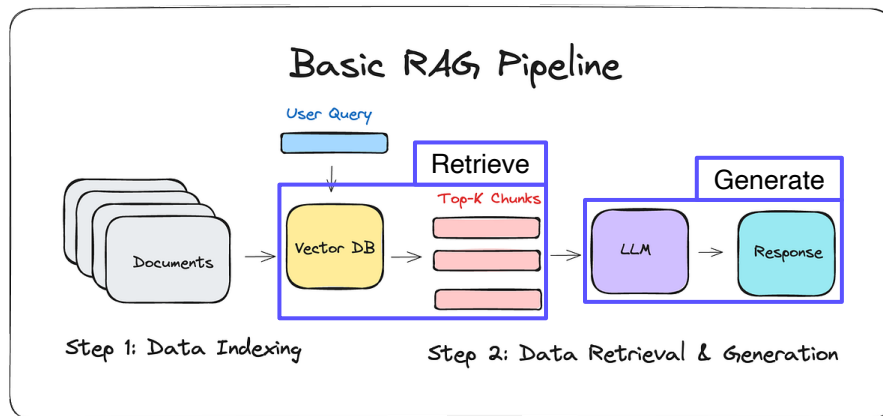


Prompt Engineering

(5) Retrieval Augmented Generation, RAG

LLM의 외부에서 신뢰할 수 있는 지식 베이스를 참조하도록 하여 추가 데이터를 제공함으로써 질문과 관련도가 높은 답변을 하도록 함. LLM이 잘못된 답변을 하는 환각(할루시네이션) 현상을 줄일 수 있음.

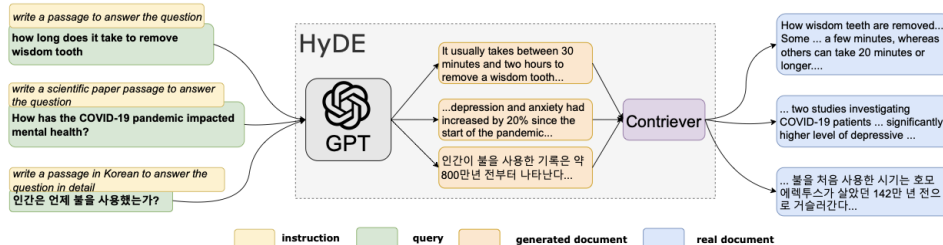
- 검색(Retriever): 질문과 문서간의 유사도를 측정하여 좋은 문서를 검색함.
- 생성(Generator): 주어진 문서를 활용하여 질문에 대한 정확하고 자연스러운 답변을 생성함.



Prompt Engineering

(5) RAG: HyDE

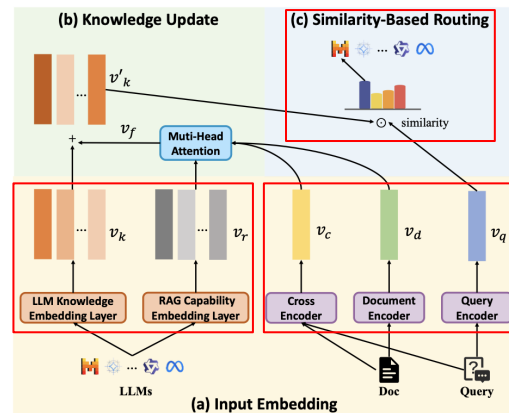
- Query Transformation의 대표 사례
- User의 Query를 바탕으로 **가상의 문서** 생성
: Query → Document
 - Zero-shot Dense Retrieval이 가능하도록 라벨이 없으면 답변을 만들어 검색하도록 함
- 생성된 가상 문서를 Dense Encoder로 임베딩하여 검색에 활용
- 가상의 Document를 만들어 RAG Retrieval 성능을 효과적으로 향상시킨 대표적 기법 중 하나
 - Dense Retrieval은 ‘문서 임베딩’을 기준으로 학습, Query를 문서 형태로 변환하면 검색 공간과의 정합성이 높아짐



Prompt Engineering

(5) RAG: RAGRouter

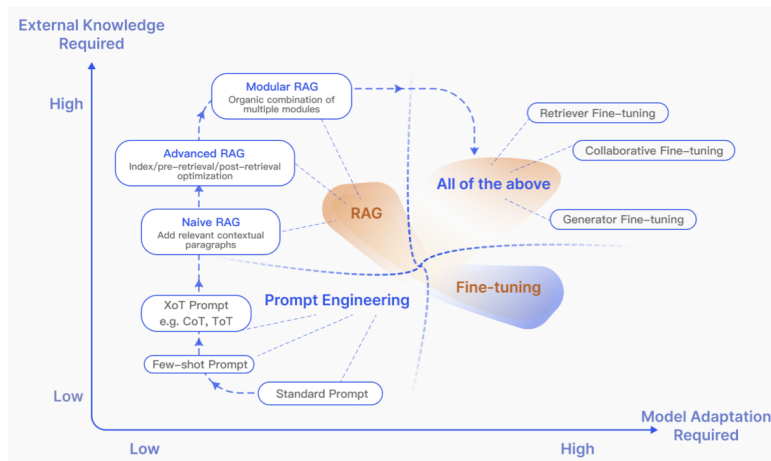
- 문서 임베딩: 검색된 지식의 맥락 파악
- LLM 지식 및 RAG 능력 임베딩:
 - 특정 LLM이 해당 해당 주제에 대해 원래 알고 있는 지식의 양을 표현
 - 해당 모델이 검색된 문서를 얼마나 정확하게 활용하여 답변을 생성할 수 있는지(능력치)를 학습
- 유사도 기반 라우팅: 학습된 지식/능력 벡터와 현재의 질의/문서 벡터 간의 유사도를 계산하여, 가장 높은 성능을 낼 수 있는 최적의 모델로 작업을 배정



(5) RAG: Modular RAG

- **Modular RAG 단계:** 고정된 흐름을 탈피하여, 특정 기능을 가진

'모듈'들을 레고 블록처럼 조립하는 단계. 문제의 성격에 따라 검색 모듈을 교체하거나 추론 루프를 추가하는 등 고도의 적응성을 제공함



(5) RAG: Modular RAG - Self-RAG

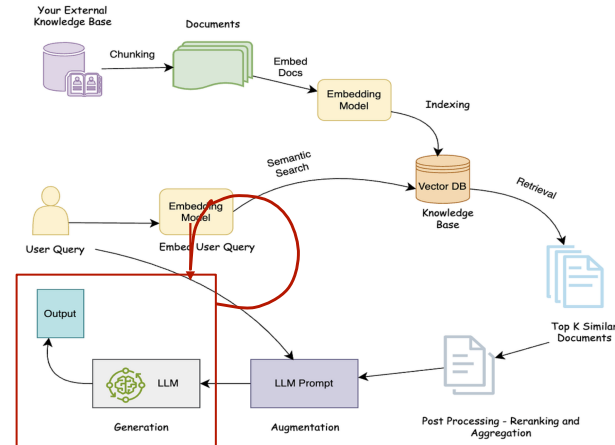
Self-RAG는 모델이 답변을 생성하면서 동시에 자신의 상태를 나타내는

특수 토큰을 내뱉도록 학습시킴

- 토큰:

- **Retrieve**: "지금 검색이 필요한가?"를 결정
- **Is-Rel (Relevance)**: "가져온 문서가 질문에 도움이 되는가?"를 판단
- **Is-Sup (Supported)**: "생성한 답변이 문서의 증거에 기반하는가?"
(현실 부합성)를 체크
- **Is-Use (Utility)**: "최종 답변이 유용한가?"를 5단계로 평가

→ 특수 토큰에 따라 검색 판단, 병렬 생성, 자기 비판 등의 단계를 통해 최종 출력 생성

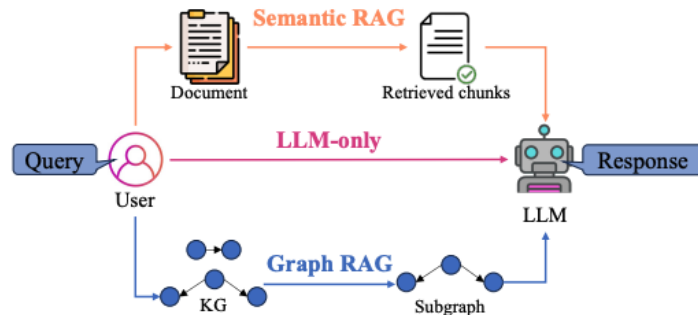


Reflect & Critic

(5) RAG: Modular RAG - Graph RAG

GraphRAG는 텍스트를 단순 나열이 아닌 지식 그래프로 변환 (e.g. Microsoft)

- 그래프 구축:
텍스트에서 개체와 관계를 추출하여 거대한 네트워크를 만듦
 - Query를 엔티티와 관계로 매핑
- 커뮤니티 탐지:
복잡하게 얽힌 그래프에서 서로 밀접한 관련이 있는 노드들을 그룹(커뮤니티)으로 묶음
- 계층적 요약:
각 커뮤니티별로 "이 그룹은 무엇에 관한 내용인가"를 미리 요약
- Querying 과정
 - Global Search: 미리 만들어둔 '커뮤니티 요약본'들을 훑으며 전체적인 맥락에서 답변
 - Local Search: 특정 개체와 연결된 이웃 노드들을 추적하며 구체적인 정보를 검색

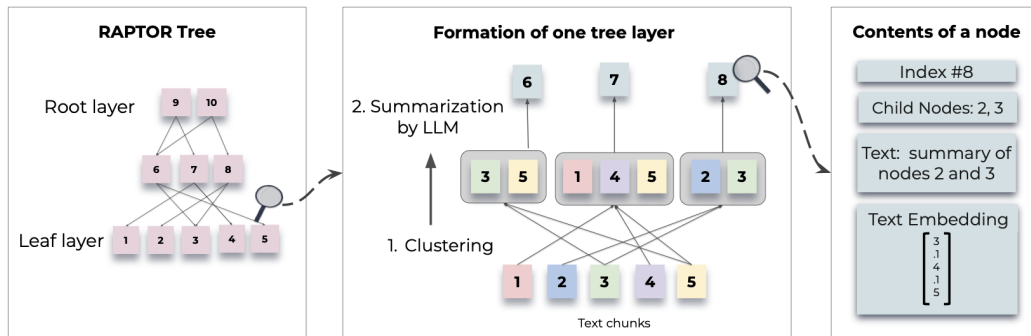


Prompt Engineering

(5) RAG: RAPTOR

작은 정보들을 묶어서 요약하고, 그 요약본들을 다시 묶어서 더 큰 요약본을 만드는 재귀적 과정 -> 긴 문서를 처리하는데 유용

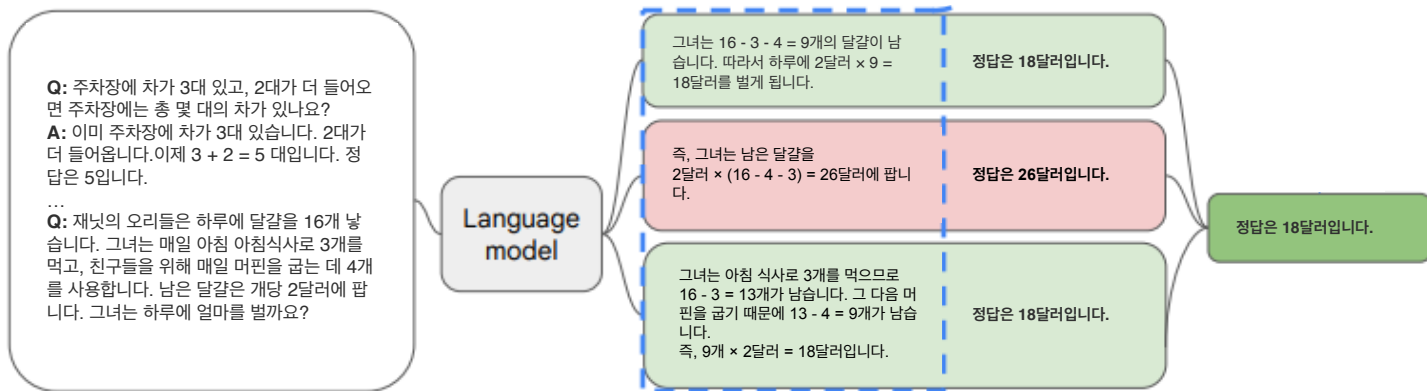
- **계층적 군집화 (Hierarchical Clustering)**: 의미적으로 유사한 텍스트 조각들을 모델(GMM 등)을 통해 그룹화
- **추상적 요약 (Abstractive Summarization)**: 묶인 조각들을 LLM에 넣어 하나의 요약된 '부모 노드'를 생성
- **트리 구조 (Tree Construction)**: 이 과정을 반복하여, 가장 아래에는 원본 청크(Leaf)가 있고, 위로 갈수록 더 넓은 범위를 아우르는 요약본(Root)이 존재하는 트리 구조를 완성



Prompt Engineering

(6) 후처리: Self-Consistency

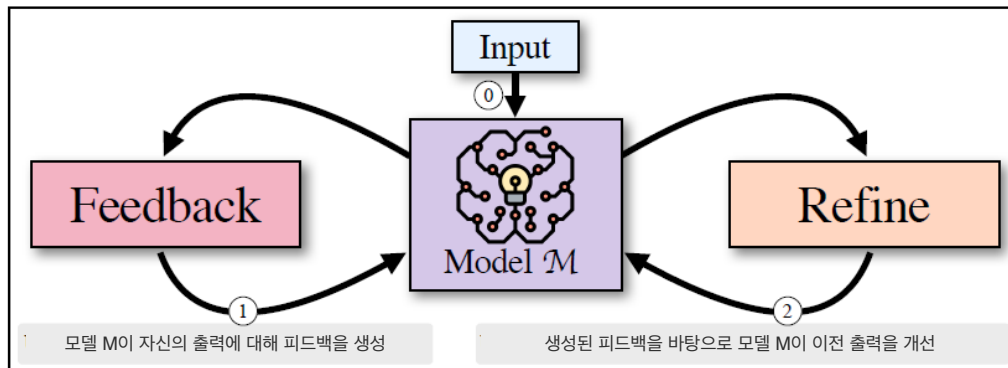
- Temperature 샘플링으로 추론 과정 포함한 여러 답변을 생성하고, 다수결을 통해 가장 일관된 답변을 선택하는 방법



Prompt Engineering

(6) 후처리: Self-Refine

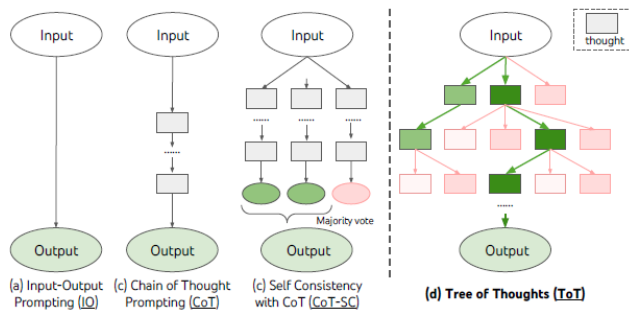
- 초기 출력에 대한 피드백을 활용하여 모델이 스스로 출력을 개선하는 방법
- 하나의 LLM을 생성기(generator), 피드백 제공자(feedback provider), 수정기(refiner)로 동시에 사용



Prompt Engineering

(6) 후처리: Tree-of-Thoughts (ToT)

- 자기 평가를 포함해 Chain of Thought(CoT) 접근을 일반화한 방법
- 사고(thought)로 구성된 추론 경로로 트리를 만들고, 그 트리에서 탐색 및 자기평가를 통해 유망한 추론 경로를 선택하는 방법
 - 사고(thought) : 추론 과정을 구성하는 일관된 텍스트 단위
- CoT 한계를 개선한 방법
 - CoT 한계1: 국소적으로, thought 과정 내에서 서로 다른 이어짐을 탐색하지 않음
 - CoT 한계2: 전역적으로, 다양한 선택지를 평가하기 위한 어떤 형태의 계획, 선택, 백트래킹도 통합하지 않음



Agentic AI

스스로 계획을 세우고 도구를 사용하여 일을 처리하는 AI

Prompt Engineering - Agentic AI

프롬프트 엔지니어링을 통해 LLM의 가능성을 확인

- 기존 목표 (프롬프트 엔지니어링): LLM에게 '어떻게 더 잘 질문할 것인가'?
- 다음 목표: '어떻게 스스로 실행 / 자동화하게 할 것인가'의 단계.
- **핵심 흐름:**
 1. Prompt Engineering (반응형): 정교한 프롬프트에 따른 콘텐츠 생성
 2. AI Agent (목표 지향형): 도구와 계획이 결합된 작업 자동화
 3. Agentic AI (자율 진화형): 스스로 목표를 설정하고 학습하는 완전 자율 시스템 (여러 Agent의 동적 결합)

Prompt Engineering - Agentic AI

[Stage 1] LLM & Prompt Engineering

- 구조: Input → Model → Output
- 핵심 특징:
 - Stateless(무상태성): 각 입력은 독립적이며, 이전 대화의 맥락을 장기적으로 기억하지 못함.
 - 반응형 AI: 사용자가 입력한 프롬프트에만 즉각적으로 반응.
- Prompt Engineering의 역할:
 - 모델의 패턴 인식 능력을 극대화하여 최적의 출력을 끌어내는 '조율사' 역할.
- 한계점: 스스로 목표를 설정하거나 복잡한 문제를 다단계로 해결하는 '자율성' 부재.

Prompt Engineering - Agentic AI

[Stage 2] AI Agents: 도구를 든 AI

- 고도화된 프롬프트 기법으로 AI 에이전트를 구성 가능.
- 특정 제한적인 태스크(Task)를 수행하는 '똑똑한 실행 단위'.
- 에이전트의 구성 요소:
 - 논리(Prompt): LLM에게 부여된 실행 지침과 역할.
 - Chain of Thought(CoT): 단계별 사고를 유도하는 프롬프트.
 - ReAct(Reasoning + Acting): "생각하고 행동하라"는 논리 구조의 주입.
 - RAG 및 도구 호출 기법
 - 도구(Tools): 검색, API, 계산기 등 프롬프트로 호출되는 기능.
 - 단기 메모리: 현재 작업을 수행하기 위한 맥락 유지.

Prompt Engineering - Agentic AI

[Stage 3] Agentic AI: 에이전트들의 '동적 오케스트레이션'

- 개별 에이전트를 조율하는 자율 시스템
- 동적 결합(Dynamic Coupling):
 - 고정된 워크플로우가 아닌, 상위 지능(Orchestrator)이 상황에 맞춰 필요한 에이전트들을 실시간으로 조합.
 - 예: "시장 분석 에이전트"와 "보고서 작성 에이전트"를 필요에 따라 연결 및 지휘.
- 시스템적 관점: 단일 에이전트가 해결 못하는 **복잡한 목표**를 분해하고 하위 에이전트에게 할당.
- 에이전트 간의 협업과 피드백 루프를 통해 결과물을 완성

Summary

Prompt Engineering

- 강의 내용 전반적인 요약
 - Prompt Engineering
 - 고급 Prompt Engineering
- Agentic AI



Building intelligence for the future of work